

Frontend Data Structure Analysis

Frontend Data Structure Analysis

Scope

This POC is a single-page Next.js App Router UI with no backend integration yet. The home screen is driven entirely by a local synthetic dataset in [src/data/listings.ts](/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/data/listings.ts:1), rendered by [src/components/home/home-page.tsx](/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/components/home/home-page.tsx:32) and [src/components/home/stay-map.tsx](/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/components/home/stay-map.tsx:59).

The frontend currently needs three categories of data:

1. Program-level listing data for cards, map pins, and popup content.
2. City-level aggregate data for the zoomed-out map state.
3. View/UI state for selection and viewport-dependent sorting/grouping.

Current UI Architecture

The page has two visual modes controlled by map zoom in [home-page.tsx](/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/components/home/home-page.tsx:30):

- `program mode`
 - Trigger: `zoom >= 5.6`
 - UI shows individual program cards and one price marker per program.
- `city mode`
 - Trigger: `zoom < 5.6`
 - UI shows one card per city and one map marker per city with the number of programs.

The map viewport is captured in [stay-map.tsx](/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/components/home/stay-map.tsx:72) and sent back to the page as `MapViewport`. That viewport is then used to:

- determine which listings are inside the visible map bounds
- sort visible listings before off-screen listings
- compute `visibleListingsCount`
- derive `citySummaries`
- switch between city mode and program mode

Exact Program-Level Data Needed

The base frontend contract is the `Listing` type in [src/data/listings.ts](/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/data/listings.ts:1).

```
``ts
type Listing = {
  id: number;
  title: string;
  location: string;
  city: string;
  state: string;
  distance: string;
  dates: string;
  price: number;
  rating: number;
  tag: string;
  image: string;
  lat: number;
  lng: number;
};
...

```

How each field is used

I Field	I Used in UI	I Purpose
---	---	---
I `id`	I cards, marker selection, active popup	I stable identifier
I `title`	I card title, popup title	I program name
I `location`	I card subtitle, popup subtitle	I combined city/state label
I `city`	I city grouping	I aggregate key
I `state`	I city cards	I city aggregate subtitle

I `distance` I card body I currently not a distance, but descriptive supporting text I
I `dates` I card body I human-readable schedule string I
I `price` I card footer, map pin label, popup I contribution amount I
I `rating` I card header I rating badge I
I `tag` I image badge I marketing label I
I `image` I card image, city card image I hero image I
I `lat` I map marker, viewport filtering I latitude I
I `lng` I map marker, viewport filtering I longitude I

Important observation

`distance` is semantically misnamed. In the current dataset it stores strings like `Guided by trained Isha teachers`, not a measurable distance. If this POC becomes a real API, that field should be renamed to something like `subtitle`, `programNote`, or `supportingText`.

Minimal JSON shape for current UI

```
```json
{
 "id": 1,
 "title": "Inner Engineering Program",
 "location": "Coimbatore, Tamil Nadu",
 "city": "Coimbatore",
 "state": "Tamil Nadu",
 "distance": "Guided by trained Isha teachers",
 "dates": "May 3-5",
 "price": 3500,
 "rating": 4.7,
 "tag": "Residential",
 "image": "https://images.unsplash.com/...",
 "lat": 10.9568,
 "lng": 76.8758
}
```

### ## Derived City-Level Data Needed

The zoomed-out UI does not read a separate API source today. It derives `CitySummary[]` from `Listing[]` in [home-page.tsx] (/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/components/home/home-page.tsx:82).

```
```ts
type CitySummary = {
  city: string;
  state: string;
  count: number;
  latitude: number;
  longitude: number;
  avgPrice: number;
  image: string;
};
```

How each city field is used

Field	Used in UI	Purpose
`city`	city card title, marker label	display name and grouping key
`state`	city card subtitle	secondary location label
`count`	city card badge, marker count	number of programs in current map view
`latitude`	city marker	marker placement
`longitude`	city marker	marker placement
`avgPrice`	city card footer	average contribution
`image`	city card image	representative image

Derivation rules in the current app

- only listings inside current map bounds are included
- grouping key is `listing.city`
- `count` increments for each listing in that city
- `avgPrice` is recalculated as a rounded running average
- first visible listing in the city supplies `latitude`, `longitude`, and `image`

Important observation

Because the grouping key is only `city`, the frontend assumes city names are unique enough by themselves. For production data, grouping should use a stronger key such as `cityId` or `city + state`.

View and Interaction State Needed

These structures are not content data, but the frontend depends on them to render correctly.

Map viewport

Defined in [stay-map.tsx](/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/components/home/stay-map.tsx:9).

```
``ts
type MapViewport = {
  latitude: number;
  longitude: number;
  zoom: number;
  bounds: {
    north: number;
    south: number;
    east: number;
    west: number;
  };
};
``
```

Used for:

- visible-result counting
- in-bounds filtering
- list sorting by proximity to map center
- city/program mode switching

Selection state

```
``ts
type UIState = {
  activeListingId: number;
  isCityMode: boolean;
  viewport: MapViewport | null;
};
``
```

Used for:

- active card styling
- active marker styling
- popup content
- deciding whether cards represent programs or cities

Data Actually Rendered by Screen Area

Header

Static labels only. No backend data required right now.

Filter chips

The chip list in [home-page.tsx](/Users/goutamsamal/Documents/isha-workspace/demo-map-project/src/components/home/home-page.tsx:21) is hardcoded:

- `Inner Engineering`
- `Hatha Yoga`
- `Residential`
- `Beginner`
- `Advanced`
- `Weekend`

These are not connected to `Listing[]` behavior yet. If the frontend is expected to support real filtering, it will need structured filter metadata instead of static strings.

Program card grid

Each card uses:

- ``image``
- ``tag``
- ``title``
- ``location``
- ``rating``
- ``distance``
- ``dates``
- ``price``

Program map markers

Each marker uses:

- ``id``
- ``lat``
- ``lng``
- ``price``

Active popup

The popup uses:

- ``title``
- ``location``
- ``price``

City card grid

Each city card uses:

- ``image``
- ``city``
- ``state``
- ``count``
- ``avgPrice``

City map markers

Each city marker uses:

- ``latitude``
- ``longitude``
- ``count``
- ``city``

Recommended API-Oriented Structure

If this POC becomes a real frontend contract, the cleanest structure is:

```
``ts
type Program = {
  id: string;
  title: string;
  cityId: string;
  city: string;
  state: string;
  locationLabel: string;
  supportingText: string;
  startDate: string | null;
  endDate: string | null;
  datesLabel: string;
  price: {
    amount: number;
    currency: "INR";
    label: string;
  };
  rating: number | null;
  tag: string | null;
  imageUrl: string;
```

```

coordinates: {
  lat: number;
  lng: number;
};
};

type CityAggregate = {
  cityId: string;
  city: string;
  state: string;
  coordinates: {
    lat: number;
    lng: number;
  };
  programCount: number;
  averagePrice: {
    amount: number;
    currency: "INR";
  };
  imageUrl: string | null;
};

type FilterOption = {
  id: string;
  label: string;
  type: "programType" | "format" | "level";
};

type HomePagePayload = {
  programs: Program[];
  cityAggregates?: CityAggregate[];
  filters: FilterOption[];
  defaultViewport: {
    latitude: number;
    longitude: number;
    zoom: number;
  };
};
```

```

### Why this shape is better

- `programs` remains the single source of truth
- `cityAggregates` can be precomputed by backend for scale, but can stay optional for the POC
- `price` and `coordinates` are grouped semantically
- `datesLabel` preserves design freedom while `startDate` and `endDate` enable real filtering later
- `cityId` removes ambiguity in grouping and selection
- `filters` becomes data-driven instead of hardcoded UI text

## Suggested Payload for This Specific POC

For the current app, this payload is enough:

```

```json
{
  "programs": [
    {
      "id": "1",
      "title": "Inner Engineering Program",
      "cityId": "coimbatore-tamil-nadu",
      "city": "Coimbatore",
      "state": "Tamil Nadu",
      "locationLabel": "Coimbatore, Tamil Nadu",
      "supportingText": "Guided by trained Isha teachers",
      "datesLabel": "May 3-5",
      "price": {
        "amount": 3500,
        "currency": "INR",
        "label": "Rs. 3500 contribution"
      },
      "rating": 4.7,
      "tag": "Residential",
    }
  ]
}
```

```

```

 "imageUrl": "https://images.unsplash.com/...",
 "coordinates": {
 "lat": 10.9568,
 "lng": 76.8758
 }
 },
 "filters": [
 { "id": "inner-engineering", "label": "Inner Engineering", "type": "programType" },
 { "id": "hatha-yoga", "label": "Hatha Yoga", "type": "programType" },
 { "id": "residential", "label": "Residential", "type": "format" },
 { "id": "beginner", "label": "Beginner", "type": "level" },
 { "id": "advanced", "label": "Advanced", "type": "level" },
 { "id": "weekend", "label": "Weekend", "type": "format" }
],
 "defaultViewport": {
 "latitude": 22.9734,
 "longitude": 78.6569,
 "zoom": 4.4
 }
}
...

```

## ## Current Gaps and Risks

1. The app has no backend contract yet; all data is local and generated in memory.
2. `distance` is misleading and should be renamed before an API is finalized.
3. Filters are only visual chips right now and do not map to query parameters or dataset fields.
4. City aggregates depend on viewport and are recomputed client-side; that is fine for a POC but not ideal for larger datasets.
5. Dates are stored as display strings only, which blocks reliable sorting and filtering.
6. Image URLs are external Unsplash links; production data should define image ownership and fallback behavior.

## ## Bottom Line

For this UI, the frontend fundamentally needs one program-level array with identifiers, labels, price, image, and coordinates. Everything else on the page is either:

- derived from that program list
- static UI chrome
- transient interaction state

If you want to formalize the frontend contract now, use `programs + filters + defaultViewport` as the API payload and treat city summaries as a derived or optional precomputed layer.